

Guía Avanzada: Especificación de Requerimientos

Proyecto: Sistema de Gestión Académica con Control Transaccional

Tecnologías: Java (OOP), SQLite, Ubuntu Server

La redacción de un Documento de Especificación de Requerimientos (SRS por sus siglas en inglés) es un paso crítico en la ingeniería de software. Para un sistema que gestiona procesos académicos sensibles empleando transacciones estrictas (ACID), la precisión en este documento define el éxito de la arquitectura y la integridad de los datos.

Paso 1: Definición del Contexto y Alcance

Antes de listar requerimientos, el equipo debe delimitar exactamente qué hará el sistema y quiénes interactuarán con él. Un enfoque avanzado requiere definir los límites del control transaccional desde el inicio: el sistema no solo "guarda datos", sino que asegura que procesos combinados (como asignar un cupo y registrar el pago) se ejecuten como una unidad indivisible.

Directriz Arquitectónica Inicial:

Debido al uso de **SQLite** (un motor embebido que bloquea a nivel de archivo), el documento debe contemplar desde el inicio restricciones de concurrencia. El sistema debe diseñarse considerando accesos serializados en las escrituras críticas.

Mapeo de Actores Principales

Actor / Rol	Nivel de Acceso	Interacciones Transaccionales Clave
Estudiante	Limitado / Lectura parcial	Inscripción de asignaturas (Alta concurrencia)
Docente	Intermedio / Escritura en grupo	Ingreso y modificación de calificaciones
Administrador	Total / Gestión integral	Cierre de ciclo académico, consolidación de datos

Paso 2: Especificación de Requerimientos Funcionales

Los requerimientos funcionales dictan lo que el sistema *debe hacer*. En un entorno Java OOP, estos requerimientos guiarán la creación de las clases de servicio y los controladores. Para redactarlos de forma avanzada, utilice el estándar de casos de uso acompañados de su validación de estado en la base de datos.

Matriz de Requerimientos Funcionales (Ejemplo)

ID	Módulo	Descripción del Requerimiento	Regla de Negocio / Restricción
RF-01	Matriculación	El sistema permitirá al estudiante inscribir hasta un máximo de 6 asignaturas por semestre.	Validación previa de prerequisites aprobados y cupos disponibles en el grupo.
RF-02	Calificaciones	El docente podrá registrar y actualizar notas mediante una planilla digital para su grupo asignado.	Bloqueo de edición automático 48 horas después del cierre oficial del periodo.
RF-03	Auditoría	El sistema registrará automáticamente (mediante Triggers de SQLite) cualquier modificación de notas.	Debe guardar: usuario, fecha, hora, nota anterior y nueva nota.
RF-04	Notificación	El sistema enviará una alerta automática al correo del estudiante al finalizar un proceso de matrícula.	Debe generar un PDF adjunto como constancia oficial.

Integración con el Diseño Orientado a Objetos:

Cada requerimiento funcional debe ser traducible a un método dentro de un objeto en Java. Por ejemplo, el RF-01 requiere la interacción entre las clases Estudiante, Asignatura y MatriculaManager. Redacte los funcionales evitando detallar el "cómo" técnico, enfocándose en el "qué" requiere el negocio.

Paso 3: Definición de Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad del sistema. Siguiendo el modelo estándar de Sommerville, se dividen en tres grandes categorías. Para este proyecto, el énfasis principal es la integridad de datos bajo el entorno específico de infraestructura.

1. De Producto (Calidad)

- **Confiabilidad (ACID):** Todas las operaciones de matriculación deben cumplir las normas de Atomicidad y Consistencia sin excepción.
- **Eficiencia:** El tiempo de respuesta para consultas de calificaciones no debe superar los 3 segundos.
- **Seguridad:** Las contraseñas en SQLite deben almacenarse utilizando un algoritmo de hash (ej. bcrypt).

2. Organizacionales (Entorno)

- **Entorno Operativo:** El sistema se ejecutará como un servicio demonio (systemd) en Linux Ubuntu Server LTS.
- **Lenguaje:** Desarrollo estricto en Java, aplicando patrones de diseño (ej. DAO, Singleton para la conexión DB).
- **Arquitectura:** Uso exclusivo de SQLite como motor relacional embebido.

3. Externos (Regulatorios y Éticos)

Privacidad de Datos: El sistema debe restringir el acceso a la base de datos física (.db). Al estar SQLite almacenado como un archivo en Ubuntu, los permisos del sistema de archivos (CHMOD) deben restringirse exclusivamente al usuario que ejecuta la aplicación Java. No se permite la exportación de listados masivos de datos personales sin el rol de Administrador de Sistema.

Evaluación Cuantitativa: Todo requerimiento no funcional debe ser medible. Evite frases ambiguas como "el sistema debe ser rápido". En su lugar, especifique tiempos exactos, porcentajes de disponibilidad o normativas legales precisas que deben superarse durante las fases de pruebas.

Paso 4: Documentación del Control Transaccional

Dada la naturaleza del sistema académico, las transacciones requieren un sub-capítulo especial. Es imperativo documentar cómo el código Java manejará las interrupciones o fallos durante una inserción múltiple para evitar bases de datos corruptas o información huérfana.

Flujo de Documentación Transaccional Requerido:

```
// El documento debe exigir lógicas similares a esta en Java:  
connection.setAutoCommit(false); try { // Operación 1: Restar cupo en SQLite //  
Operación 2: Insertar registro de matrícula // Operación 3: Actualizar historial  
del estudiante connection.commit(); // Éxito total } catch (SQLException e) {  
connection.rollback(); // Falla: Reversión total }
```

Paso 5: Validación, Trazabilidad y Aprobación

El paso final en la elaboración del documento es asegurar que cada requerimiento definido tenga una forma clara de ser probado (Trazabilidad hacia casos de prueba). Se debe incluir una matriz formal de firmas para el congelamiento de alcance.

Fase de Validación	Responsable	Firma y Fecha
Revisión de Lógica de Negocio	Director Académico	_____
Revisión de Arquitectura (Java/SQLite)	Líder Técnico de Proyecto	_____
Aprobación Final para Desarrollo	Gerente del Proyecto	_____